

MyCloud — Architecture & Build Plan

Sep 30, 2026 · @Moheddine

Overview

MyCloud is a multi-tenant personal cloud: users register, log in with 2FA, create spaces, and upload, preview and download images, videos and files. The backend is Go microservices on a single-node k3s cluster on the home mini-PC, served at `cloud.moheddine.dev`.

Fixed constraints that drive the design:

- **IPv6-only home connection (Telekom DS-Lite):** Cloudflare Tunnel is required, and no inbound ports are opened.
- **Domain `moheddine.dev` is on Cloudflare:** DNS, WAF, DDoS protection and Turnstile come from one place.
- **One Fujitsu Q958 mini-PC:** single node now, second node later, so resource limits matter.
- **Cloudflare free plan caps request bodies at 100 MB:** uploads must be chunked.
- **Admin access over Tailscale only:** kubectl and SSH are never public.

Tech map

The stack is Go + React on k3s, with S3-compatible object storage for files and Postgres for metadata only.

Area	Technology	Purpose
Backend	Go: <code>net/http</code> (1.22+ routing) or <code>chi</code> , <code>pgx</code> + <code>sqlc</code> , <code>goose</code> , <code>oapi-codegen</code>	APIs and workers; <code>sqlc</code> gives type-safe, parameterized SQL
Frontend	React + TypeScript (Vite), TanStack Query, React Router, Uppy	Cloud UI with resumable uploads
API contract	OpenAPI 3 specs in <code>api/</code>	Single source of truth for server and client code

Area	Technology	Purpose
Metadata DB	PostgreSQL via CloudNativePG operator	Users, spaces, files, quotas (one DB per service)
Object storage	Garage or SeaweedFS (S3 API)	Actual file bytes; check MinIO community edition status before choosing it
Sessions / cache	Redis	Sessions, rate limits, cache
Events	NATS JetStream	Durable async pipeline between services
Uploads	tus protocol (tusd Go library)	Resumable chunked uploads under the 100 MB edge limit
Auth	Go + argon2id + TOTP (pquerna/otp)	Login, 2FA, recovery codes
Images	libvips (govips)	Thumbnails, EXIF stripping
Video	FFmpeg / ffprobe (Intel QuickSync via VAAPI)	Poster frames, metadata, optional transcoding
Documents	pdftoppm or MuPDF	PDF thumbnails, text preview
Malware	ClamAV	Scan every upload
Containers	Docker, distroless base images	Packaging
Orchestration	k3s (single node, Longhorn when the second node arrives)	Platform
Ingress	Traefik	Routing, ForwardAuth, rate limits, headers
Edge	Cloudflare DNS, WAF, DDoS, Bot Fight Mode, Turnstile	Protect the origin
Exposure	Cloudflare Tunnel (cloudflared in cluster)	Public access with zero open ports
Admin access	Tailscale	Private kubectl/SSH
Secrets	SOPS + age (or Sealed Secrets)	Encrypted secrets in Git

Area	Technology	Purpose
Metrics / dashboards	kube-prometheus-stack, Grafana	Measurements and alerts
Logs	Loki + Grafana Alloy	Centralized logs
Tracing	OpenTelemetry SDK + Tempo	Request tracing
CI	GitHub Actions, golangci-lint, Trivy, Renovate	Build, test, scan, update
Registry	GHCR	Container images
CD	Argo CD + Kustomize	GitOps deployment
Backups	restic to Hetzner Storage Box or Backblaze B2; CNPG backups	Off-site copies, tested restores
Email	Resend, Postmark, or SMTP on <code>mohedine.dev</code>	Verification and password reset
Load testing	k6	Verify limits before sharing

Microservices

Seven services: four APIs and three event-driven workers. Each service owns its own database; none reads another's tables.

Service	Type	Responsibility	Owns
<code>auth-service</code>	API	Register, login, TOTP 2FA, recovery codes, sessions, invite codes, Traefik ForwardAuth check	<code>auth</code> DB, Redis sessions
<code>space-service</code>	API	Space CRUD, plans, quota reserve/commit/release, usage	<code>spaces</code> DB
<code>upload-service</code>	API	tus endpoint, quota reservation, streams chunks to S3 multipart, publishes <code>file.uploaded</code>	stateless
<code>file-service</code>	API	Metadata, listing, rename, delete, download streaming with HTTP Range	<code>files</code> DB

Service	Type	Responsibility	Owns
scan-worker	Worker	ClamAV scan, marks file ready or quarantined	none
media-worker	Worker	Image thumbnails, EXIF/GPS stripping, video posters, optional transcode	none
doc-worker	Worker	PDF first-page thumbnail, text preview	none

Communication: the browser uses REST/JSON through Traefik, and services exchange events over NATS (file.uploaded → file.scanned → file.ready). gRPC between services is an optional later learning step.

Architecture

All public traffic enters through Cloudflare and an outbound tunnel. The object store, database and queue are reachable only inside the cluster.

```

flowchart LR
    U[Browser / React SPA] -->|HTTPS| CF[Cloudflare<br/>DNS · WAF · DDoS · Bot Fight · Turnstile]
    CF -->|outbound tunnel| CFD[ccloudflared pod]
    subgraph k3s["k3s single node (Q958 mini-PC)"]
        CFD --> T[Traefik<br/>rate limit · headers · ForwardAuth]
        T --> FE[frontend nginx]
        T --> AUTH[auth-service]
        T --> SPACE[space-service]
        T --> UP[upload-service tus]
        T --> FILE[file-service]
        UP --> S3[(Garage / S3)]
        FILE --> S3
        UP -- file.uploaded --> NATS1[NATS JetStream]
        NATS1 --> SCAN[scan-worker + ClamAV]
        SCAN -- file.scanned --> NATS2[NATS]
        NATS2 --> MEDIA[media-worker]
        NATS2 --> DOC[doc-worker]
        MEDIA --> S3
        DOC --> S3
        AUTH --> R[(Redis)]
        AUTH --> PG1[(PostgreSQL CNPG)]
        SPACE --> PG2[PG]
    end

```

```

FILE --> PG
end
ADMIN[Admin] -->|Tailscale| k3s
PG -.backup.-> OFF[(Off-site backup)]
S3 -.restic.-> OFF

```

Routing is single-origin on `cloud.mohedine.dev`: `/` serves the frontend, and `/api/auth`, `/api/spaces`, `/api/uploads`, `/api/files` go to the services. That means no CORS and simple cookies.

Space model

A space is a logical container owned by one user. It is a UUIDv7 `space_id` plus a prefix in one shared bucket, not a separate bucket or database.

Identity: `space_id` (UUIDv7), display name, and a slug unique per owner. URLs use the ID: `/spaces/{space_id}`.

Storage layout: keys are built from IDs only, never from user filenames. That blocks path traversal and name collisions.

```

mycloud-data/
  spaces/{space_id}/files/{file_id}/original
  spaces/{space_id}/files/{file_id}/thumb_256.webp
  spaces/{space_id}/files/{file_id}/poster.jpg

```

Isolation: every query filters on `space_id` and checks `owner_id = current_user`. Other users' spaces return 404, not 403. Postgres row-level security comes later as defense in depth.

Quota: 15 GB per account, shared across all of a user's spaces, with a maximum of 5 spaces. Limits live in a `plans` table, so scaling up is a data change.

1. **Reserve** on upload start: `used + reserved + size ≤ quota`, checked in one transaction.
2. **Commit** on completion: the reserved bytes become used bytes.
3. **Release** on abort or expiry: a cleanup job handles abandoned uploads.

Table	Key columns
users	id, email, password_hash (argon2id), email_verified, totp_secret (encrypted), plan_id
recovery_codes	user_id, code_hash, used_at
invites	code, created_by, used_by
plans	id, quota_bytes, max_spaces
spaces	id, owner_id, name, slug, created_at
usage	user_id, used_bytes, reserved_bytes
files	id, space_id, name, mime, size, sha256, object_key, kind, status (uploading / scanning / ready / quarantined)
file_variants	file_id, variant (thumb / poster / preview), object_key

Upload and download flow

Uploads arrive in tus chunks under 100 MB and stream straight into S3 multipart.

Downloads are streamed by `file-service` with HTTP Range, so video seeking works and the object store never goes public.

```
sequenceDiagram
    participant B as Browser (Uppy)
    participant T as Traefik
    participant A as auth-service
    participant U as upload-service
    participant S as space-service
    participant O as Garage/S3
    participant N as NATS
    participant W as workers
    participant F as file-service
    B->>T: POST /api/uploads (tus create, size, space_id)
    T->>A: ForwardAuth (session cookie)
    A-->>T: 200 + X-User-Id
    T->>U: forward
    U->>S: reserve quota(user, space, size)
```

```

S-->U: ok
loop chunks < 100 MB
  B-->U: PATCH chunk
  U-->O: multipart part
end
U-->O: complete multipart
U-->N: file.uploaded
N-->W: scan, then process
W-->N: file.ready
B-->T: GET /api/files/{id}/content (Range)
T-->F: forward
F-->O: ranged GET
O-->B: bytes (via file-service)

```

Processing rules

- **All files:** detect type by magic bytes against an allowlist, scan with ClamAV, and store binaries in S3 only (metadata goes to Postgres).
- **Images:** strip EXIF including GPS, then generate WebP thumbnails with libvips.
- **Videos:** extract metadata with ffprobe and generate a poster frame. Keep the original; transcode to H.264 MP4 with faststart only when browsers can't play it (QuickSync). HLS comes later.
- **PDF and text:** first-page thumbnail and a text snippet.

Security plan

Security is layered from the Cloudflare edge down to the data disk. Nothing on the home network is reachable without passing through Cloudflare or Tailscale.

Layer	Controls
Edge (Cloudflare)	Proxied DNS, automatic DDoS protection, WAF managed rules, rate-limit rule on <code>/api/auth/*</code> , Bot Fight Mode, Turnstile on register and login, optional EU geo-restriction
Network	Cloudflare Tunnel with zero inbound ports, UFW deny inbound, admin only via Tailscale
Ingress (Traefik)	Rate-limit middleware, CSP / HSTS / X-Frame-Options headers, body-size limits, ForwardAuth

Layer	Controls
Cluster	Default-deny NetworkPolicies, non-root pods, read-only root filesystem, Pod Security restricted , resource limits
Application	sqlc/pgx parameterized queries against SQL injection, input validation, argon2id, TOTP + recovery codes, HttpOnly + Secure + SameSite=Strict cookies, CSRF token, account lockout, uniform auth errors
Registration	Invite codes only
Files	ClamAV scan, magic-byte MIME allowlist, quarantine status, Content-Disposition: attachment for non-media
Data	LUKS-encrypted data disk, encrypted TOTP secrets, nightly off-site backups, tested restores
Supply chain	Trivy scans, Renovate, pinned image digests, distroless images

Repository structure

One monorepo with a Go workspace (`go.work`). CI uses path filters so each service builds only when its own code changes.

```
mycloud/
  CLAUDE.md
  .claude/skills/           project skills for Claude Code
  services/
    auth/ space/ upload/ file/
    scan-worker/ media-worker/ doc-worker/
  pkg/                     shared Go libs (logging, otel, config, httpx,
events)
  web/                     React + Vite app
  api/                     OpenAPI specs
  deploy/
    base/                  Kustomize base per service
    overlays/prod/
    platform/              traefik, cloudflared, cnpg, garage, nats, redis,
observability
  docs/
    architecture.md
    adr/                   architecture decision records
  .github/workflows/
```

Roadmap

The build runs in ten phases, each ending at a checkable gate. Observability is deliberately late to save RAM on the mini-PC.

```
flowchart LR
    P0[0 Platform] --> P1[1 GitOps] --> P2[2 Data layer] --> P3[3 Auth] -->
P4[4 Spaces]
    P4 --> P5[5 Files] --> P6[6 Pipeline] --> P7[7 Hardening] --> P8[8
Observability] --> P9[9 Extras]
```

Phase	Deliverables	Done when
0 Platform	k3s, Traefik, cloudflared tunnel, LUKS data disk, Tailscale kubectl	Hello-world page loads at <code>cloud.mohedine.dev</code>
1 GitOps	Monorepo, GitHub Actions (lint, test, build, Trivy), GHCR, Argo CD, SOPS	A git push redeploys hello-world
2 Data layer	CloudNativePG, Garage, Redis, NATS, restic off-site	One backup restored successfully

Phase	Deliverables	Done when
3 Auth	Invite-based register, email verify, login, TOTP, recovery codes, ForwardAuth, login UI	2FA login works end to end
4 Spaces	Space CRUD, plans, quota reserve/commit/release, dashboard UI	Two users see only their own spaces
5 Files	tus upload, listing, Range download, delete with quota release, upload UI	A 2 GB video uploads, resumes and seeks
6 Pipeline	scan-, media- and doc-workers, previews in UI	EICAR test file is quarantined
7 Hardening	NetworkPolicies, WAF and rate rules, Turnstile, headers, pod security, k6 load test	Security checklist passes
8 Observability	kube-prometheus-stack, Loki + Alloy, Tempo, OTel in all services	One request is traceable across services
9 Extras	Share links, search, HLS, trash/restore, second node + Longhorn	Pick per interest

Working with Claude Code

Claude Code in VS Code works from the repo itself: `CLAUDE.md` holds the standing context and `.claude/skills/` holds project skills, both committed to Git.

1. Create the empty `mycloud` repo and open it in VS Code.
2. Save this doc as `docs/architecture.md` (export as Markdown).
3. Run the bootstrap prompt in Claude Code: it writes `CLAUDE.md`, the project skills, and suggests marketplace plugins.
4. Restart the Claude Code session so the new skills load, then ask it to list available skills.
5. Work one roadmap phase per session and record each decision as an ADR in `docs/adr/`.